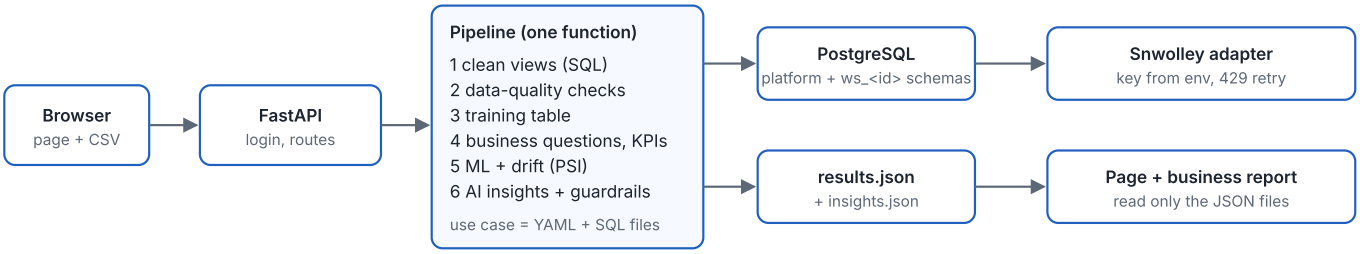
Architecture note

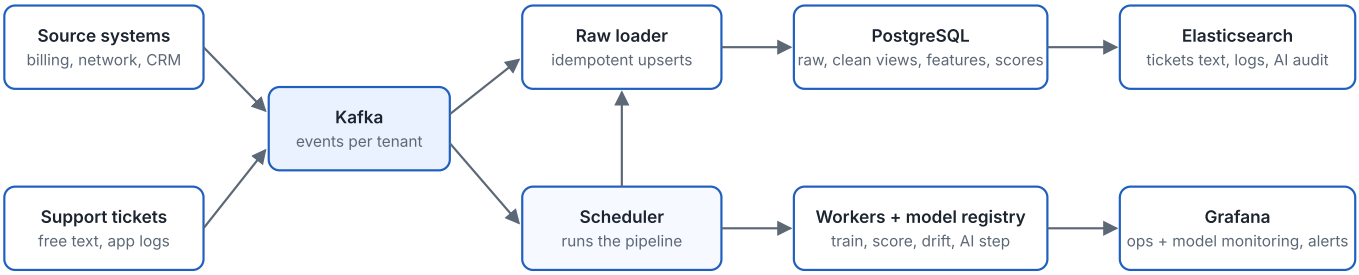
AI-augmented ML analytics platform: what is built, and how it grows into production with Kafka, Elasticsearch, Grafana and scheduled pipelines.

1. What is built (the prototype)



- **Multi-tenancy:** one Postgres schema per workspace (name chosen by the server, never the client); platform tables (users, workspaces, runs) in their own schema. A partial unique index allows only one running analysis per workspace.
- **Generic by configuration:** the platform code never mentions churn or revenue. A use case is configuration plus SQL: a developer writes them for the demo business (two ship), and for a business's own file the platform builds the same thing from the business's column choices (target, inputs, id, date). The form sends only column names; the server generates the SQL (clean view, data-quality checks, training table, business questions, KPIs) from names it created and validated at upload, so the browser never supplies SQL.
- **Two ways in:** (a) the demo business, generated by a seeded script; (b) the business's own CSV: size, encoding and shape checks, safe column names, a profile (types, gaps, common values), then loaded as text into `upload_raw` in the workspace's schema and cleaned by a view (typed, trimmed, duplicates flagged, never deleted). Limits: one flat file (10 MB, 200,000 rows); no "as of" date, so leakage is warned about rather than proven; drift is skipped until a later export date exists.
- **Traceability:** `results.json` is the single record of a run. The AI summary and the report read only that record, so every number can be traced to SQL or the ML step.
- **AI behind an adapter:** the Snwolley call lives in one file (one message per request, 5 requests/minute limit handled with a Retry-After wait and one retry). The AI only sees a numbered facts sheet; its output is checked (citations, numbers, causal wording, allowed recommendation types) and falls back to a labelled template.

2. Production architecture



Component	Role	Why, and what it is not for
Kafka	Source systems publish events (payment, usage, ticket, plan change), keyed by workspace and customer. A consumer upserts into the raw tables using the event id.	Decouples sources from the platform, buffers bursts, lets us replay history after a bug fix. Event ids make duplicates harmless, which is exactly the duplicate-row problem the clean views handle today. Not used for analytics queries.
Scheduled pipelines	Nightly: load, clean views, **data-quality gate**, features at today's date, score with the registered model, drift check, AI insights, report. Weekly or monthly: retrain as champion vs challenger. Each run is idempotent and recorded in `platform.runs`.	Orchestrator such as Airflow or Prefect, with a job queue (Celery/RQ) replacing the prototype's in-process background task. A failed gate stops scoring and keeps the last good results, marked stale.
PostgreSQL	System of record for raw data, cleaning views and features, plus predictions and run history.	SQL stays central. Schema per workspace works for hundreds of tenants; very large tenants move to their own database. Add a pooler (PgBouncer) and read replicas for reports.
Elasticsearch	Indexes what Postgres is bad at: support-ticket text (search, topic counts, a text feature for models), application and pipeline logs, and an audit trail of every AI summary with its facts and rejections.	Not the analytics store. The current data has no free text, so this is the first thing to add once ticket text is ingested; logs and the AI audit are useful immediately.
Grafana	Dashboards and alerts for the people who run the platform: run success and duration, data-quality counts, PSI per feature over time, score distribution, model metrics once labels arrive, AI accepted / rejected / fallback rates.	Reads Postgres (a monitoring table written by each run) and Elasticsearch (logs). Managers keep reading the business report; Grafana is for operators. Thresholds are in `docs/monitoring_plan.md`.

3. Cross-cutting decisions

- **Security:** uploads are size-capped while streaming, must be UTF-8 and rectangular, and are never executed; column names are rebuilt by the server, the one category value the business picks must be a value the profile saw, and only the aggregated facts sheet reaches the AI. secrets in a secrets manager, not files; the AI key is read only by the adapter and never logged. Customer names, emails and phones never reach the AI (it sees aggregated facts only). Per-tenant topics, schemas and index aliases keep tenants apart; the server, not the client, decides the tenant. Auth moves to a shared session store, with throttling in Redis rather than process memory.
- **Reproducibility:** seeds and library versions recorded in the model card; the registry stores model, training data hash and metrics, so any score can be traced to the run that produced it.
- **Failure behaviour:** if the AI provider is down or its answer fails the checks, reports still ship with the template summary, labelled. If data quality fails, scoring stops rather than producing confident nonsense.
- **Cost and scale:** feature SQL is the heavy part (the prototype fixed slow CTEs with `MATERIALIZED` and pre-aggregation); at scale features are computed incrementally per day and stored, and training runs on a worker, not the web process.

4. What I would build next

1) A monitoring table plus a Grafana dashboard fed by each run. 2) Replace the background task with a queue and scheduler (also smooths the AI rate limit). 3) Kafka consumer for one source (payments) to prove the ingestion path. 4) Time-based train/test split and performance monitoring when labels mature. 5) An "ask about this run" box for the agent. 6) Multi-table upload with a join mapping, and a time-based split plus drift for uploads that carry a date.